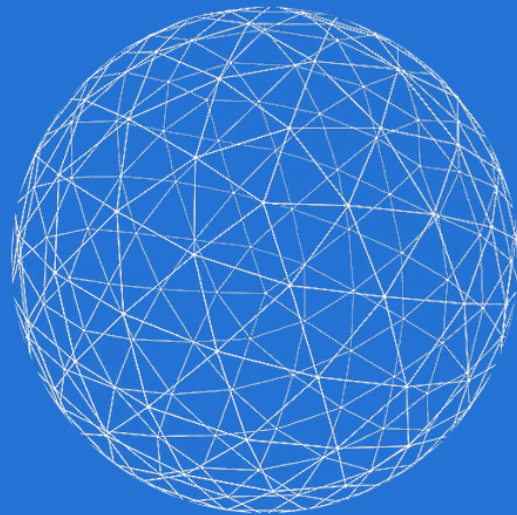




第五章 自动驾驶和避障





CONTENTS

目录

- 1 自动驾驶和避障实现步骤
- 2 卷积神经网络CNN
- 3 自动驾驶和避障实现



教学目标和重难点

➤ 教学目标

1. 掌握自动驾驶和避障的实现步骤

重点

2. 熟悉CNN网络在项目中的使用

重点

3. 自动驾驶和避障功能的代码实现

难点



① 自动驾驶和避障的实现步骤



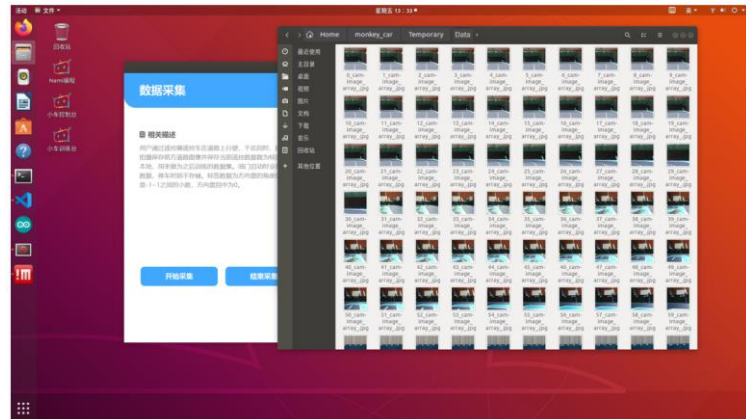
自动驾驶和避障实现步骤

➤ 1.1 实现步骤

1. 数据采集（小车通过摄像头采集图片、通过手柄采集转向角度和速度）
 - 车辆行驶中的图片
 - 车辆转向角度
 - 车辆形式速度
 - 构建模型时我们使用图片和角度两个特征值
2. 模型训练
 - 通过图像和角度两个特征值利用CNN网络训练模型
3. 模型推理
 - 输入特征为图片，经过模型推理输出角度值

➤ 1.2 数据介绍

1. 小车通过摄像头采集图片、通过手柄采集转向角度和速度
采集过程后面任务中详细介绍，在此我们直接使用。
2. txt文件记录的是图片路径、角度、速度

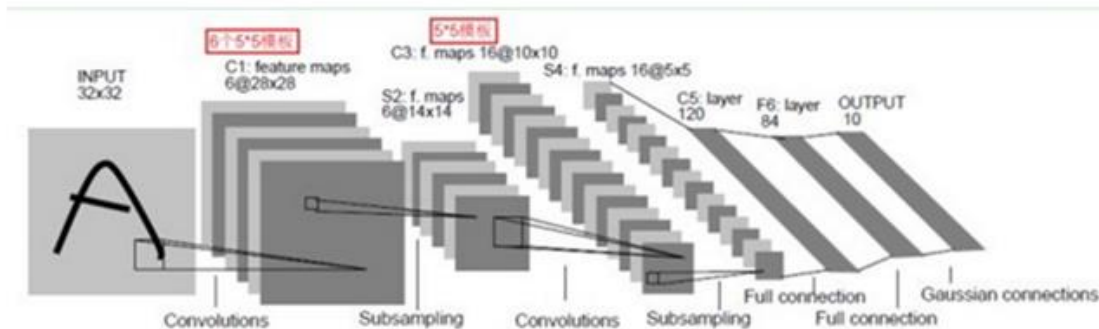




② 卷积神经网络CNN

➤ 2.1 CNN介绍

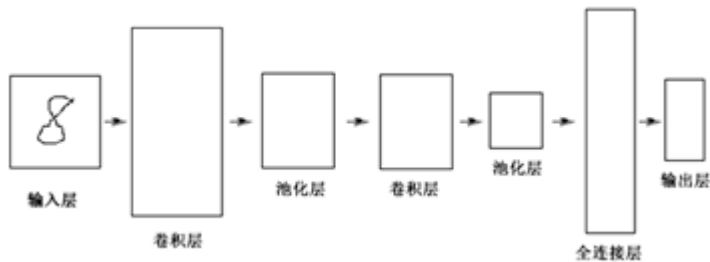
卷积神经网络 (Convolutional Neural NetWork,简称 CNN) 是一种包含卷积计算且具有深度结构的前馈神经网络, 被广泛应用于图像识别、自然语言处理和语音识别等领域。本实验中自动驾驶轨道和障碍物的识别就使用的是CNN来实现。



➤ 2.2 模型结构

CNN网络核心的层有：输入层、卷积层、池化层、全连接层、输出层。

在实际应用中各层的数量可以灵活配置。



➤ 2.3 本项目网络结构

本项目中我们构建了9层网络，包含：5层卷积、4层全连接。详细结构如下：

```
# RaKnet 神经网络
def get_classify_model(self, num_class):
    input_img = Input(shape=(conf.IMG_DEPTH, conf.IMG_HEIGHT, conf.IMG_WIDTH), name='input_img')
    if K.image_data_format() == 'channels_last': # 确认输入维度
        input_img = Input(shape=(conf.IMG_HEIGHT, conf.IMG_WIDTH, conf.IMG_DEPTH), name='input_img')
    x = input_img

    x = Conv2D(20, (5, 5), strides=(2, 2), padding='same', activation='relu')(x)
    x = Conv2D(50, (3, 3), strides=(2, 2), padding='same', activation='relu')(x)
    x = Conv2D(50, (3, 3), strides=(1, 1), padding='same', activation='relu')(x)
    x = Conv2D(80, (3, 3), strides=(2, 2), padding='same', activation='relu')(x)
    x = Conv2D(80, (3, 3), strides=(1, 1), padding='same', activation='relu')(x)

    x = Flatten(name='flattened')(x)
    x = Dense(100, activation='relu')(x)
    x = Dropout(.1)(x)
    x = Dense(50, activation='relu')(x)
    x = Dropout(.1)(x)

    angle = Dense(num_class, activation='softmax', name='angle')(x)
    throttle = Dense(1, activation='relu', name='throttle')(x)
    model = Model(inputs=[input_img], outputs=[angle, throttle])

    return model
```

经过层层的卷积，核心特征被提取出来，在全连接层实现分类任务，良好的行车路线角度被计算出来，最终实现沿着训练路线自动行驶。

2.4 数据集介绍

2084-2023-04-03.jpg	0-2023-04-03.jpg 0.0 0.07119868260750145	9 kB
2085-2023-04-03.jpg	1-2023-04-03.jpg 0.0 0.30832850123599964	27 kB
2086-2023-04-03.jpg	2-2023-04-03.jpg 0.0 0.5612964262825404	27 kB
2087-2023-04-03.jpg	3-2023-04-03.jpg 0.0 0.6640522476882229	29 kB
2088-2023-04-03.jpg	4-2023-04-03.jpg 0.0 0.6640522476882229	28 kB
2089-2023-04-03.jpg	5-2023-04-03.jpg 0.0 0.6640522476882229	25 kB
2090-2023-04-03.jpg	6-2023-04-03.jpg 0.11066011535996582 0.6640522476882229	29 kB
2091-2023-04-03.jpg	7-2023-04-03.jpg 0.2766502839991457 0.6640522476882229	28 kB
2092-2023-04-03.jpg	8-2023-04-03.jpg 0.5533310953093051 0.6640522476882229	28 kB
2093-2023-04-03.jpg	9-2023-04-03.jpg 0.8062990203550458 0.6640522476882229	25 kB
2094-2023-04-03.jpg	10-2023-04-03.jpg 0.8862990203550458 0.6640522476882229	28 kB
2095-2023-04-03.jpg	11-2023-04-03.jpg 0.4663830616071891 0.6640522476882229	28 kB
2096-2023-04-03.jpg	12-2023-04-03.jpg 0.06371101614119 0.4663830616071891	28 kB
2097-2023-04-03.jpg	13-2023-04-03.jpg 0.0 0.11066011535996582	111.0 kB
label.txt	14-2023-04-03.jpg 0.0 0.2766502839991457	68.5 kB
train.txt	15-2023-04-03.jpg 0.0 0.434827723014008	13.0 kB
val.txt	16-2023-04-03.jpg 0.0 0.5850053081453902	
	17-2023-04-03.jpg 0.0 0.00872219000824	
	18-2023-04-03.jpg 0.0 0.0245307779160866	
	19-2023-04-03.jpg 0.0 0.6403393658253731	
	20-2023-04-03.jpg 0.11066011535996582 0.6403393658253731	
	21-2023-04-03.jpg 0.30832850123599964 0.6403393658253731	
	22-2023-04-03.jpg 0.30832850123599964 0.6403393658253731	
	23-2023-04-03.jpg 0.28455458235419784 0.6403393658253731	

标签文件内容

图像名称
角度值
速度值

采集的图像

采集时生成的
标签文件

训练时分割出
训练与验证标
签文件



③ 自动驾驶和避障实现



自动驾驶和避障实现

➤ Python核心依赖库

- tensorflow==1.8.0
- Keras==2.2.4

➤ 核心步骤

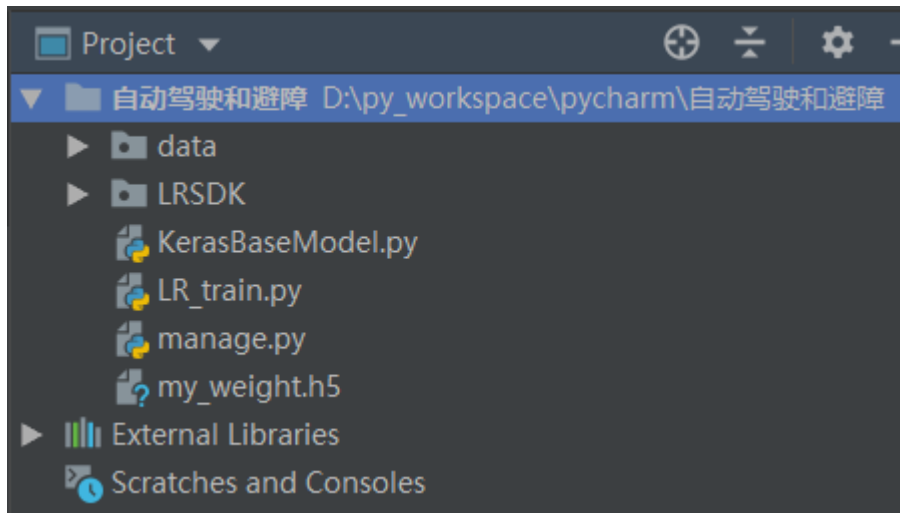
- 使用CNN构建网络模型
- Keras==2.2.4



自动驾驶和避障实现

➤ 3.1 构建项目结构

data数据集、LRSDK小车驱动包、KerasBaseModel.py基础模型、LR.train.py训练代码



➤ 3.2 使用CNN构建模型(model.py)

CNN网络包含5层卷积、4层全连接层，定义模型层：

```
# RaKnet 神经网络
def get_classify_model(self, num_class):
    input_img = Input(shape=(conf.IMG_DEPTH, conf.IMG_HEIGHT, conf.IMG_WIDTH), name='input_img')
    if K.image_data_format() == "channels_last": # 确认输入维度
        input_img = Input(shape=(conf.IMG_HEIGHT, conf.IMG_WIDTH, conf.IMG_DEPTH), name='input_img')
    x = input_img

    x = Conv2D(20, (5, 5), strides=(2, 2), padding='same', activation='relu')(x)
    x = Conv2D(50, (3, 3), strides=(2, 2), padding='same', activation='relu')(x)
    x = Conv2D(50, (3, 3), strides=(1, 1), padding='same', activation='relu')(x)
    x = Conv2D(80, (3, 3), strides=(2, 2), padding='same', activation='relu')(x)
    x = Conv2D(80, (3, 3), strides=(1, 1), padding='same', activation='relu')(x)

    x = Flatten(name='flattened')(x)
    x = Dense(100, activation='relu')(x)
    x = Dropout(.1)(x)
    x = Dense(50, activation='relu')(x)
    x = Dropout(.1)(x)

    angle = Dense(num_class, activation='softmax', name='angle')(x)
    throttle = Dense(1, activation='relu', name='throttle')(x)
    model = Model(inputs=[input_img], outputs=[angle, throttle])

    return model
```

➤ 3.3 调用模型进行训练(model.py)

- 定义模型训练方法

参数：模型、模型保存路径、训练集、测试集等

1. 选择优化器

```
# 训练
def train(self,
          model,
          save_model_path,
          train_x,
          train_angle_y,
          train_throttle_y,
          test_x,
          test_angle_y,
          test_throttle_y,
          batch_size=10,
          epochs=32,
          lr=1e-3,
          early_stop=True,
          num_workers=1,
          optimizer='adam',
          patience=10):

    # 选择优化器
    print('[ 20003 ] - Choose Optimizer...')
    if optimizer == 'adam':
        opt = Adam(lr=lr / conf.BATCH_SIZE)
    elif optimizer == 'nadam':
        opt = Nadam(lr=lr / conf.BATCH_SIZE)
    elif optimizer == 'sgd':
        opt = SGD(lr=lr / conf.BATCH_SIZE, momentum=conf.MOMENTUM, decay=conf.WEIGHT_DECAY)
    else:
        error_msg = '{0} optimizer is not support, please check you input optimizer types!'
        print('[ 20004 ] - {}'.format(error_msg))
        raise Exception(error_msg)
```



自动驾驶和避障实现

➤ 3.3 使用CNN构建模型(Model.py)

2. 训练类型

3. 展示网络结构

4. 打印参数

```
# 训练类型
print('[ 20005 ] - Model Compile...')

model.compile(optimizer=opt, metrics=["acc"],
              loss={'angle': 'categorical_crossentropy',
                    'throttle': 'mean_absolute_error'},
              loss_weights={'angle': 0.9, 'throttle': .01}
              )

# 展示网络结构
print('[ 20006 ] - Show Model Summary...')
RaKnetModel().show_model_summary(model)

# 打印参数
print('[ 20007 ] - Print Training Parameters...')
```

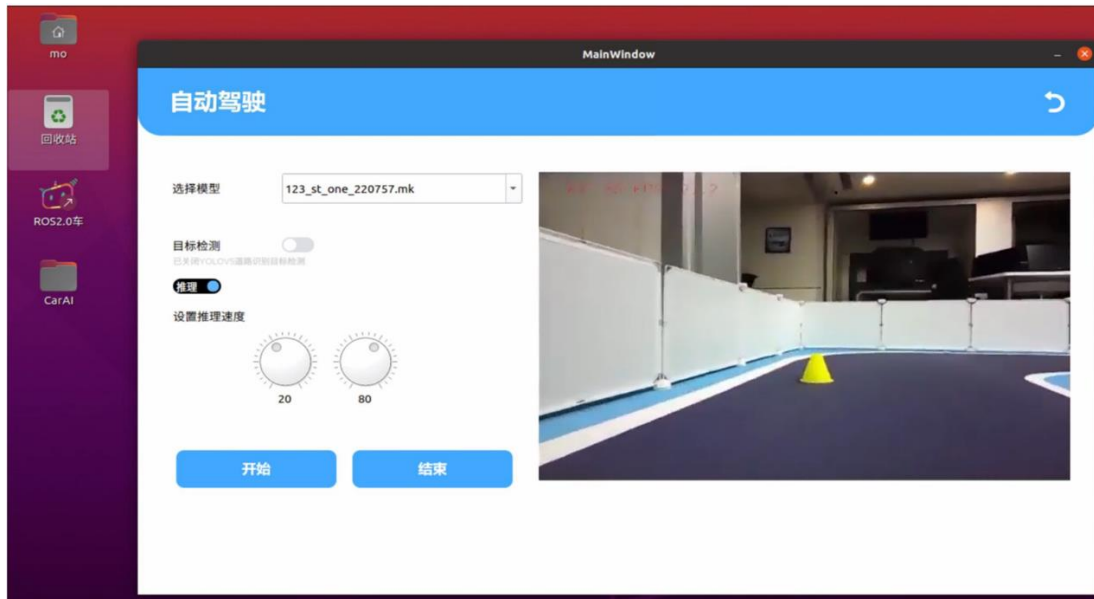
➤ 3.3 使用CNN构建模型(Model.py)

- 5. 设置early_stop

```
# 设置early_stop
if early_stop:
    # patience = int(epochs * 0.1) if epochs > 100 else 10
    print('[ 20009 ] - Early Stop, Patience : {}'.format(patience))
    patience_type = 'loss' # val_loss
    callbacks_list = [keras.callbacks.EarlyStopping(monitor=patience_type, patience=patience, verbose=0),
                     keras.callbacks.ModelCheckpoint(save_model_path, monitor=patience_type, verbose=0,
                                                     save_best_only=True, mode='min')]
    print('[ 20010 ] - Callbacks List : {}'.format(callbacks_list))
else:
    callbacks_list = [keras.callbacks.ModelCheckpoint(save_model_path, monitor='val_loss', verbose=0,
                                                     save_best_only=True, mode='min')]
    print('[ 20011 ] - Callbacks List : {}'.format(callbacks_list))
```

自动驾驶和避障实现

➤ 3.4 调用模型





总结

- 自动驾驶和避障数据集重要特征值：图片、角度
- 模型训练使用卷积神经网络CNN实现
 - 输入层、卷积层、池化层、全连接层、输出层。
 - 在实际应用中各层的数量可以灵活配置。
- 本项目构架构建了9层CNN网络
 - 5层卷积、4层全连接



Thank You

AI改变生活